Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

1480. Running Sum of 1d Array

Easy | Topics | Companies | Hint

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]…nums[i])`.

Return the running sum of `nums`.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [1,3,6,10]
Explanation: Running sum is obtained as follows: [1, 1+2, 1+2+3, 1+2+3+4].
```

Example 2:

```
Input: nums = [1,1,1,1,1]
Output: [1,2,3,4,5]
Explanation: Running sum is obtained as follows: [1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1].
```

Example 3:

```
Input: nums = [3,1,2,10,1]
Output: [3,4,6,16,17]
```

Constraints:

- `1 <= nums.length <= 1000`
- `-10^6 <= nums[i] <= 10^6`

8.9K | 221 | 142 Online

Code

Java | Auto

```java
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] = nums[i] + nums[i - 1];
        }
        return nums;
    }
}
```

Saved

Ln 8, Col

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[1,2,3,4]

Output

[1,3,6,10]

Expected

[1,3,6,10]

Contribute a testcase

Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

1672. Richest Customer Wealth

Easy | Topics | Companies | Hint

You are given an `m x n` integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return *the **wealth** that the richest customer has.*

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

> **Input:** accounts = [[1,2,3],[3,2,1]]
> **Output:** 6
> **Explanation:**
> 1st customer has wealth = 1 + 2 + 3 = 6
> 2nd customer has wealth = 3 + 2 + 1 = 6
> Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

> **Input:** accounts = [[1,5],[7,3],[3,5]]
> **Output:** 10
> **Explanation:**
> 1st customer has wealth = 6
> 2nd customer has wealth = 10
> 3rd customer has wealth = 8
> The 2nd customer is the richest with a wealth of 10.

Example 3:

> **Input:** accounts = [[2,8,7],[7,1,3],[1,9,5]]
> **Output:** 17

Constraints:

- `m == accounts.length`

4.9K | 110

58 Online

Code

Java | Auto

```java
class Solution {
    public int maximumWealth(int[][] accounts) {
        int maxWealth = 0;

        for (int[] customer : accounts) {
            int wealth = 0;

            for (int money : customer) {
                wealth += money;
            }

            maxWealth = Math.max(maxWealth, wealth);
        }

        return maxWealth;
    }
}
```

Saved

Ln 17, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

accounts =
[[1,2,3],[3,2,1]]

Output

6

Expected

6

Contribute a testcase

Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

977. Squares of a Sorted Array

Easy | Topics | Companies

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of* ***the squares of each number*** *sorted in non-decreasing order.*

Example 1:

> **Input:** nums = [-4,-1,0,3,10]
> **Output:** [0,1,9,16,100]
> **Explanation:** After squaring, the array becomes [16,1,0,9,100].
> After sorting, it becomes [0,1,9,16,100].

Example 2:

> **Input:** nums = [-7,-3,2,3,11]
> **Output:** [4,9,9,49,121]

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an `O(n)` solution using a different approach?

Discover more | Computer Science

Seen this question in a real interview before? 1/6

Yes | No

10.4K | 195

124 Online

Code

Java | Auto

```java
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        int left = 0;
        int right = n - 1;
        int index = n - 1;

        while (left <= right) {
            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];

            if (leftSquare > rightSquare) {
                result[index] = leftSquare;
                left++;
            } else {
                result[index] = rightSquare;
                right--;
            }

            index--;
        }

```

Saved

Ln 27, Col 2

Testcase | Test Result

nums =

[-4,-1,0,3,10]

Output

[0,1,9,16,100]

Expected

[0,1,9,16,100]

Contribute a testcase

Problem List

Description | Editorial | Solutions | Submissions

724. Find Pivot Index

Easy | Topics | Companies | Hint

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is `0` because there are no elements to the left. This also applies to the right edge of the array.

Return *the **leftmost pivot index***. If no such index exists, return `-1`.

Example 1:

> **Input:** nums = [1,7,3,6,5,6]
> **Output:** 3
> **Explanation:**
> The pivot index is 3.
> Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
> Right sum = nums[4] + nums[5] = 5 + 6 = 11

Example 2:

> **Input:** nums = [1,2,3]
> **Output:** -1
> **Explanation:**
> There is no index that satisfies the conditions in the problem statement.

Example 3:

> **Input:** nums = [2,1,-1]
> **Output:** 0
> **Explanation:**
> The pivot index is 0.
> Left sum = 0 (no elements to the left of index 0)
> Right sum = nums[1] + nums[2] = 1 + -1 = 0

9.5K | 277 | 119 Online

Code

Java | Auto

```java
int totalSum = 0;

// Calculate total sum
for (int num : nums) {
    totalSum += num;
}

int leftSum = 0;

// Find pivot index
for (int i = 0; i < nums.length; i++) {
    int rightSum = totalSum - leftSum - nums[i];

    if (leftSum == rightSum) {
        return i;
    }

    leftSum += nums[i];
}

return -1;
}
```

Saved | Ln 25, Col

Testcase | Test Result

Input

nums =
[1,7,3,6,5,6]

Output

3

Expected

3

Contribute a testcase